# AURUM

AI Finance Intelligence Dashboard

Part 2 of 2 — **Codebase & File Guide**

What every important file does, and how the folders fit together.

Prepared 14 July 2026 · Author: Bhanu Prakash

# 0 · The folder map

The project is a single Next.js app. There is no separate backend — the API route handlers *are* the backend. Here's the whole tree with the important files:

```
AI-Finance Dashboard/
├─ prisma/
│   ├─ schema.prisma        # the database blueprint (all tables)
│   └─ seed.ts              # generates 2 years of fake books
├─ src/
│   ├─ app/                   # routes (App Router: folder = URL)
│   │   ├─ layout.tsx         # root shell, fonts, providers
│   │   ├─ page.tsx           # "/" landing page
│   │   ├─ login/ signup/     # auth pages
│   │   ├─ dashboard/         # /dashboard
│   │   ├─ predictions/       # /predictions
│   │   ├─ insights/          # /insights
│   │   ├─ transactions/      # /transactions
│   │   └─ api/               # backend endpoints
│   │       ├─ auth/[...nextauth]/  # NextAuth handler
│   │       ├─ register/      # create account
│   │       ├─ insights/      # run the AI analyst
│   │       └─ transactions/  # paginated ledger
│   ├─ components/            # React UI (charts, tables, shell)
│   ├─ lib/                   # the "brain": db, auth, finance, AI
│   └─ types/                 # TypeScript type augmentation
├─ .env                      # secrets (DB url, auth secret, API key)
├─ package.json              # dependencies + npm scripts
└─ next.config.ts / tsconfig.json / eslint.config.mjs
```

**The one rule to remember:** nothing touches the database or Claude directly except code in `src/lib/`. Pages and API routes call into `lib`; `lib` talks to PostgreSQL (via Prisma) and to Claude (via the Anthropic SDK). That's the whole architecture.

# 1 · The database layer (prisma/)

DATABASE BLUEPRINT

### `prisma/schema.prisma`

**The single most important file for understanding the data.** It declares the datasource (PostgreSQL) and every table as a "model". Prisma reads this file to (a) generate a fully typed query client and (b) create the real tables via `prisma db push`.

The models (tables):

- `User` — the operator who logs in (email, bcrypt password hash).
- `MonthlyFinancial` — one row per month; the P&L backbone every chart reads (revenue, expenses, operational/non-operational split).
- `ExpenseByCategory` — expense amount per month per category (feeds the donut).
- `Product` — catalogue with unit economics (price, cost, units sold, rating).
- `Transaction` — the 520-row ledger; links to a Product; has status + method enums.
- `Campaign` — marketing campaigns vs a revenue target.
- `InsightBatch` → `Insight` — one AI generation run (summary + health score + N finding items). A one-to-many relation.

Also defines the enums: `TxStatus`, `TxMethod`, `CampaignStatus`, `InsightSeverity`, `InsightSource`.

DATA GENERATOR

### `prisma/seed.ts`

Fills an empty database with a believable two-year story. Run with `npm run db:seed`. Key techniques worth mentioning in an interview:

- **Deterministic PRNG** (`mulberry32` with a fixed seed) — every reseed produces *identical* data, so screenshots and demos are reproducible.
- **Realistic P&L** — base revenue $182K compounding at ~2.8%/month, multiplied by a Q4-gifting seasonality curve and random noise; expense ratio slowly improves from 78% → ~66% as the brand "scales".
- **Weighted transactions** — a `sqrt` curve biases the 520 orders toward recent months (more mass at the recent end), mimicking a growing business.
- **Idempotent** — wipes tables in dependency order first, so it can be re-run safely. Also creates the `demo@aurum.finance` login.

## 2 · The brain (src/lib/)

These files hold all the logic. Everything else just calls into them.

DATABASE CLIENT

### src/lib/prisma.ts

Creates and exports one shared `PrismaClient`. Uses a global-singleton trick so that Next.js hot-reloading in dev doesn't spawn a new DB connection on every save (which would exhaust the connection pool). Every query in the app imports `prisma` from here.

AUTH CONFIGURATION

### src/lib/auth.ts

The NextAuth config object. Defines the **Credentials provider** whose `authorize()` looks up the user, `bcrypt.compare()`s the password, and returns the user on success. Sets `session.strategy = "jwt"` and uses callbacks to smuggle the user `id` into the token and back onto the session. Signed with `AUTH_SECRET`.

SESSION HELPER

### src/lib/session.ts

A tiny convenience wrapper: `getSession()` = `getServerSession(authOptions)`. Every protected page/route calls this to read the current user on the server.

DASHBOARD DATA ACCESS

### src/lib/finance.ts

All the **read queries** the dashboard needs: `getMonthlyFinancials`, `getExpenseCategories`, `getProducts`, `getRecentTransactions`, `getCampaigns`. Each fetches from Prisma and maps the rows into **plain-JSON shapes** (dates → ISO strings) so they can be safely passed from server components down to client chart components. Also exports the shared TypeScript row types.

THE AI ANALYST

### src/lib/insights.ts

**The most important file after the schema.** Contains the entire AI pipeline:

- `buildSnapshot()` — rolls raw books into KPIs + z-score anomaly detection.
- `computeHealthScore()` — the deterministic 0–100 score (never from the AI).
- `heuristicInsights()` — the templated fallback analyst (no API key needed).
- `aiInsights()` — calls Claude via the Anthropic SDK using a **tool definition** and forced `tool_choice` for guaranteed-structured JSON; returns `null` on any failure so the fallback kicks in.
- `generateAndSaveInsights()` — orchestrates the above and persists an `InsightBatch`.

THE FORECAST MATH

**src/lib/regression.ts**

Pure math, no database, no dependencies. `linearRegressionForecast()` implements ordinary-least-squares from scratch: computes slope + intercept, $R^2$ ( `1-SSE/SST` ), residual σ, a ±1.96σ confidence band, the 12-month projection, and average MoM growth. Returns a tidy result the Predictions view renders directly.

FORMATTERS

**src/lib/utils.ts**

Shared display helpers: `formatMoney` (exact, for tables), `formatMoneyCompact` ($3.75M / $264K, for tiles & charts), `formatPercent`, and the month/date label formatters. Small but used everywhere for consistent number formatting.

# 3 · Routes & pages (src/app/)

In the App Router, **a folder is a URL** and its `page.tsx` is what renders. Files here are server components unless marked `"use client"`.

| FILE | PURPOSE |
|---|---|
| `app/layout.tsx` | Root layout wrapping every page: loads the Manrope + Playfair Google fonts, applies global CSS, and mounts the `<Providers>` (NextAuth session context). Sets the page title/meta. |
| `app/page.tsx` | The "/" landing page. Reads the session; if you're already signed in it redirects straight to `/dashboard`. |
| `app/login/page.tsx` | Client component. The sign-in form; calls NextAuth's `signIn("credentials")`, then routes to the dashboard. |
| `app/signup/page.tsx` | Client component. The create-account form; POSTs to `/api/register`. |
| `app/dashboard/page.tsx` | Server component. Guards the session, then `Promise.all`-fetches all five datasets from `lib/finance` and hands them to `<DashboardShell>`. Marked `force-dynamic` so it always reflects live data. |
| `app/predictions/page.tsx` | Server component. Pulls monthly revenue, builds 12 future month labels, runs `linearRegressionForecast`, and passes the result to `<PredictionsView>`. |
| `app/insights/page.tsx` | Server component. Loads the latest saved `InsightBatch` from the DB and renders `<InsightsView>` (which can trigger a fresh generation). |
| `app/transactions/page.tsx` | Server component shell that renders `<LedgerView>`, which then fetches its own data client-side. |

## API routes (the backend)

| FILE | ENDPOINT & JOB |
|---|---|
| `app/api/auth/[...nextauth]/route.ts` | The catch-all NextAuth handler — powers `/api/auth/*` (sign-in, sign-out, session). Just wires up `authOptions`. |
| `app/api/register/route.ts` | `POST /api/register` — validates email + password (min 8 chars), rejects duplicates, bcrypt-hashes the password, and creates the `User`. |
| `app/api/insights/route.ts` | `POST /api/insights` — session-guarded; calls `generateAndSaveInsights()` to run the analyst and persist a batch. |
| `app/api/transactions/route.ts` | `GET /api/transactions` — session-guarded; builds a Prisma `where` from query/status/page params, returns `{ total, rows }` with server-side pagination. |

# 4 · UI components (src/components/)

The visual layer. Chart and interactive components are client components; they receive plain JSON as props and render it.

| FILE | PURPOSE |
|---|---|
| `components/Shell.tsx` | The app chrome: the fixed left sidebar (nav links + user + sign-out) on desktop, top bar on mobile. Wraps every signed-in page. |
| `components/Providers.tsx` | Client wrapper that mounts NextAuth's `<SessionProvider>` so client components can read session state. |
| `components/Monogram.tsx` | The little gold "A" logo mark, reused on auth pages and the sidebar. |
| `components/dashboard/DashboardShell.tsx` | The dashboard's brain on the client: holds the range filter state and, in one `useMemo`, scopes every dataset + computes the KPI deltas, then lays out all the panels. |
| `components/dashboard/StatTile.tsx` | A single KPI tile: big value, coloured delta (respects `upIsGood`), and a sparkline. |
| `components/dashboard/PnlCharts.tsx` | The P&L visuals: Revenue-vs-Expenses area, Revenue-by-month bar, Net-profit line, Operational-vs-non-operational lines. |
| `components/dashboard/CompositionCharts.tsx` | The expense-composition donut, the product unit-economics scatter, and the campaign progress meters. |
| `components/dashboard/Tables.tsx` | The "Recent transactions" and "Top products" tables, plus the reusable `StatusBadge`. |
| `components/charts/chartKit.tsx` | The shared chart "design system": the validated colour palette (`VIZ`), tooltip, legend, axis/grid styling, and the `ChartCard` wrapper. Ensures every chart looks like one system. |
| `components/predictions/PredictionsView.tsx` | Renders the forecast: the 4 model tiles, the actuals/fit/forecast composed chart with the confidence band, and the forecast detail table. Owns the "Show forecast" toggle. |
| `components/insights/InsightsView.tsx` | Renders the AI output: executive summary, health meter, and the severity-tagged finding cards. The "Generate/Regenerate" button POSTs to `/api/insights` then refreshes. |
| `components/transactions/LedgerView.tsx` | The interactive ledger: debounced search, status filter, pagination, and CSV export — all driven by fetches to the transactions API. |

# 5 · Config & supporting files

| FILE | PURPOSE |
|---|---|
| `.env` | Secrets & config: `DATABASE_URL` (PostgreSQL), `AUTH_SECRET` / `NEXTAUTH_SECRET` (JWT signing), `NEXTAUTH_URL`, `ANTHROPIC_API_KEY` (enables real Claude), optional `INSIGHTS_MODEL`. Never committed. |
| `.env.example` | A template of the above with blank/placeholder values, safe to commit — shows teammates what to fill in. |
| `package.json` | Declares dependencies and the npm scripts: `dev`, `build`, `start`, `lint`, `db:seed`, and a `postinstall` that runs `prisma generate`. |
| `src/types/next-auth.d.ts` | TypeScript augmentation that adds `id` to the NextAuth `Session.user` type, so `session.user.id` is type-safe across the app. |
| `src/app/globals.css` | Global styles & the theme tokens (the gold/ink colour variables, the dark "page-glow" background) used by Tailwind utility classes. |
| `next.config.ts` | Next.js configuration (currently minimal/default). |
| `tsconfig.json` | TypeScript compiler settings, including the `@/*` path alias that maps to `src/`. |
| `eslint.config.mjs` / `postcss.config.mjs` | Linting rules (via `eslint-config-next`) and the Tailwind/PostCSS pipeline. |

# 6 · The commands (cheat sheet)

| COMMAND | WHAT IT DOES |
|---|---|
| `npm install` | Installs deps; auto-runs `prisma generate` after. |
| `npx prisma db push` | Creates/updates the DB tables from `schema.prisma`. |
| `npm run db:seed` | Fills the DB with the 2-year demo dataset. |
| `npm run dev` | Starts the dev server at `localhost:3000`. |
| `npx prisma studio` | Opens a GUI to browse the tables (`localhost:5555`). |
| `npm run build` / `start` | Production build / run. |

**Prerequisite:** the PostgreSQL 17 service (`postgresql-x64-17`) must be running before `db push`, `db:seed`, or `dev` — otherwise you'll see "Can't reach database server at localhost:5432".